

Wildfire Analytics Final Report

Project Overview and Goals

This project is an end-to-end analytics data pipeline, specifically focused on wildfire and weather data in the United States. Data is updated daily for an interactive dashboard that will show users daily weather and wildfire data, specifically at the county level. This serves as a resource for Americans to see if their local county or a county of loved ones is at risk for wildfires.

There were three initial goals, first, to create an interactive dashboard for users' benefit, second to analyze what climactic factors lead to high levels of wildfires, and third to predict wildfire risk level across the United States. These goals had to be paired down due to project complexity and time constraints, so in the end the deliverables are a simplistic wildfire risk level calculation and an interactive dashboard.

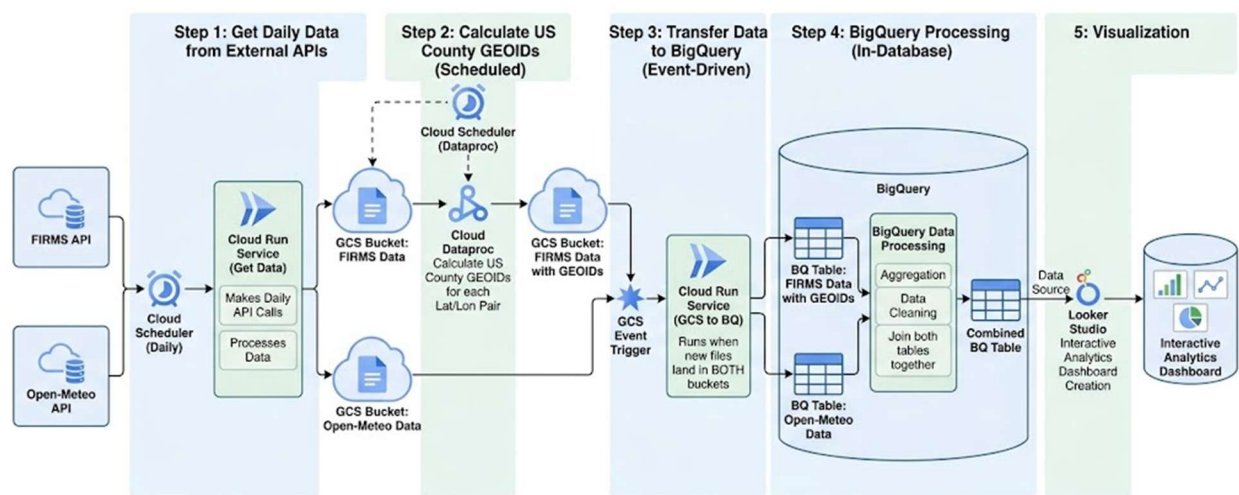
Dataset Description

Two main data sources were utilized in this project. The first was the FIRMS NASA API, which provides near real time active fire data. Specifically, this project utilized data from the Visible Infrared Imaging Radiometer Suite (VIIRS) which comes from multiple satellites that effectively try to find active wildfires based on thermal anomalies. This is based on mid infrared radiation levels and longwave channel readings. On the project's end, we get daily sets of latitude and longitude pairs that are the locations of suspected wildfires along with confidence level of the location actually being a wildfire, brightness level, and fire radiative power. The total daily dataset size depends on the day. For example, in the winter you might have hundreds of data points since fires are uncommon in the winter, where in the summer you might have hundreds of thousands.

The project also utilizes weather data from the Open-Meteo API. This API allows you to input sets of latitude and longitude pairs and get daily weather data for the given location such as

temperature, precipitation, and wind levels. This weather data is important to the project because wildfire risk is heavily dictated by weather conditions. If there is a lot of rain, risks for wildfires lowers. The catch for this API is that it has fairly harsh limits for API calls, with a maximum of 10,000 API calls per day. This may seem high, but each lat/lon pair is considered a single API call. The API also throttles API calls based on finer grain metrics such as hourly and by the minute. For reference, there are over 3000 counties in the US, so it is quite easy to hit the daily/hourly limits on this API.

Pipeline Workflow and Services Used



Pipeline Workflow Steps:

1. Data Ingestion:

- Cloud Run services with imbedded Python script are built to make API calls and pull data.
 - The Open-Meteo Cloud Run service specifically utilizes a single lat/lon pair from each US county, which allows the weather data to already have county geoid's attached to the data.
- Cloud Scheduler has a daily ticker, which runs both Cloud Run services at 2 AM.
- The new data is stored in separate GCS buckets by the Cloud Run services.

2. Data Processing using Dataproc

- Open-Meteo data already contains county geoid through the Cloud Run ingestion, but the FIRMS data's lat/lon pairs need have their county calculated based on county geographic

boundaries. So Dataproc is utilized to do this calculation. The new FIRMS data with county geoids is added to a new GCS bucket. This Dataproc workflow is scheduled at 3 AM (after data ingestion at 2 AM) through Cloud Scheduler.

3. **BigQuery and Data Processing**

- Cloud Run services are again utilized to do data transfer from the GCS buckets to BigQuery tables.
- A trigger is implemented so whenever a new file is added to the buckets, the service is triggered, which appends the data to a BigQuery table. There are individual tables for both datasets.

4. **BigQuery and Data Transformation**

- Datasets are cleaned and joined using BigQuerySQL. This is implemented using a scheduled query that runs daily.

5. **Data Visualization**

- Looker Studio is utilized through data connection to the final BigQuery table to provide an interactive dashboard and a finished product.

Data Processing Steps:

- Open-Meteo data is fairly clean going in and does not require adding county geoid because the lat/lon pairs that are sent to the API are generated based on the center of each US county.
- Open-Meteo data is filtered to get distinct rows in the case that the same data was appended to a bucket multiple times.
- The datatypes from the FIRMS Dataproc job are all converted to strings, so the date attribute is casted to a date type, the county geoid is casted to an integer, and the radiation and brightness values are casted to floats.
- The FIRMS data is cleaned by removing any suspected wildfires that have a confidence rating of low. The only suspected wildfires that remain are of medium or high confidence.
- FIRMS data is then grouped by date and county geoid, and the total number of suspected wildfires is calculated per county along with average brightness and radiation.
- The two datasets are then joined on based on geoid and date.

- The final product is for each geoid and date pair, there is a total number of suspected wildfires, the average intensity of the wildfires, and the daily weather.
- Lastly, the rudimentary wildfire risk index metric is calculated for each county and date. This is a simple weighted norm, with weights multiplied to each feature based on perceived importance to wildfire risk. As seen below, the normalization of each feature is calculated, which is a calculation that takes $\text{norm}(x) = (x - \min(x)) / (\max(x) - \min(x))$. This puts everything on a 0-1 scale. These norms are then multiplied by weights, and get summed to produce an index value where a higher value is a higher risk for wildfires. With more time, I would have calculated these weights more accurately using Machine Learning. More historic data would have also been extremely useful, but was not practical due to time constraints.

```

45 normalized AS (
46     SELECT
47         *,
48         SAFE_DIVIDE(temp_max_f - MIN(temp_max_f) OVER(),
49                     MAX(temp_max_f) OVER() - MIN(temp_max_f) OVER()) AS n_temp_max,
50         SAFE_DIVIDE(wind_gusts_max_mph - MIN(wind_gusts_max_mph) OVER(),
51                     MAX(wind_gusts_max_mph) OVER() - MIN(wind_gusts_max_mph) OVER()) AS n_wind_gust,
52         SAFE_DIVIDE(sunshine_duration_s - MIN(sunshine_duration_s) OVER(),
53                     MAX(sunshine_duration_s) OVER() - MIN(sunshine_duration_s) OVER()) AS n_sunshine,
54         SAFE_DIVIDE(IFNULL(num_fires, 0) - MIN(IFNULL(num_fires, 0)) OVER(),
55                     MAX(IFNULL(num_fires, 0)) OVER() - MIN(IFNULL(num_fires, 0)) OVER()) AS n_fires,
56         SAFE_DIVIDE(IFNULL(avg_frp, 0) - MIN(IFNULL(avg_frp, 0)) OVER(),
57                     MAX(IFNULL(avg_frp, 0)) OVER() - MIN(IFNULL(avg_frp, 0)) OVER()) AS n_frp,
58         SAFE_DIVIDE(IFNULL(avg_bright, 0) - MIN(IFNULL(avg_bright, 0)) OVER(),
59                     MAX(IFNULL(avg_bright, 0)) OVER() - MIN(IFNULL(avg_bright, 0)) OVER()) AS n_bright,
60         SAFE_DIVIDE(rain_sum_in - MIN(rain_sum_in) OVER(),
61                     MAX(rain_sum_in) OVER() - MIN(rain_sum_in) OVER()) AS n_rain,
62         SAFE_DIVIDE(temp_min_f - MIN(temp_min_f) OVER(),
63                     MAX(temp_min_f) OVER() - MIN(temp_min_f) OVER()) AS n_temp_min
64     FROM combined
65 )

```

```

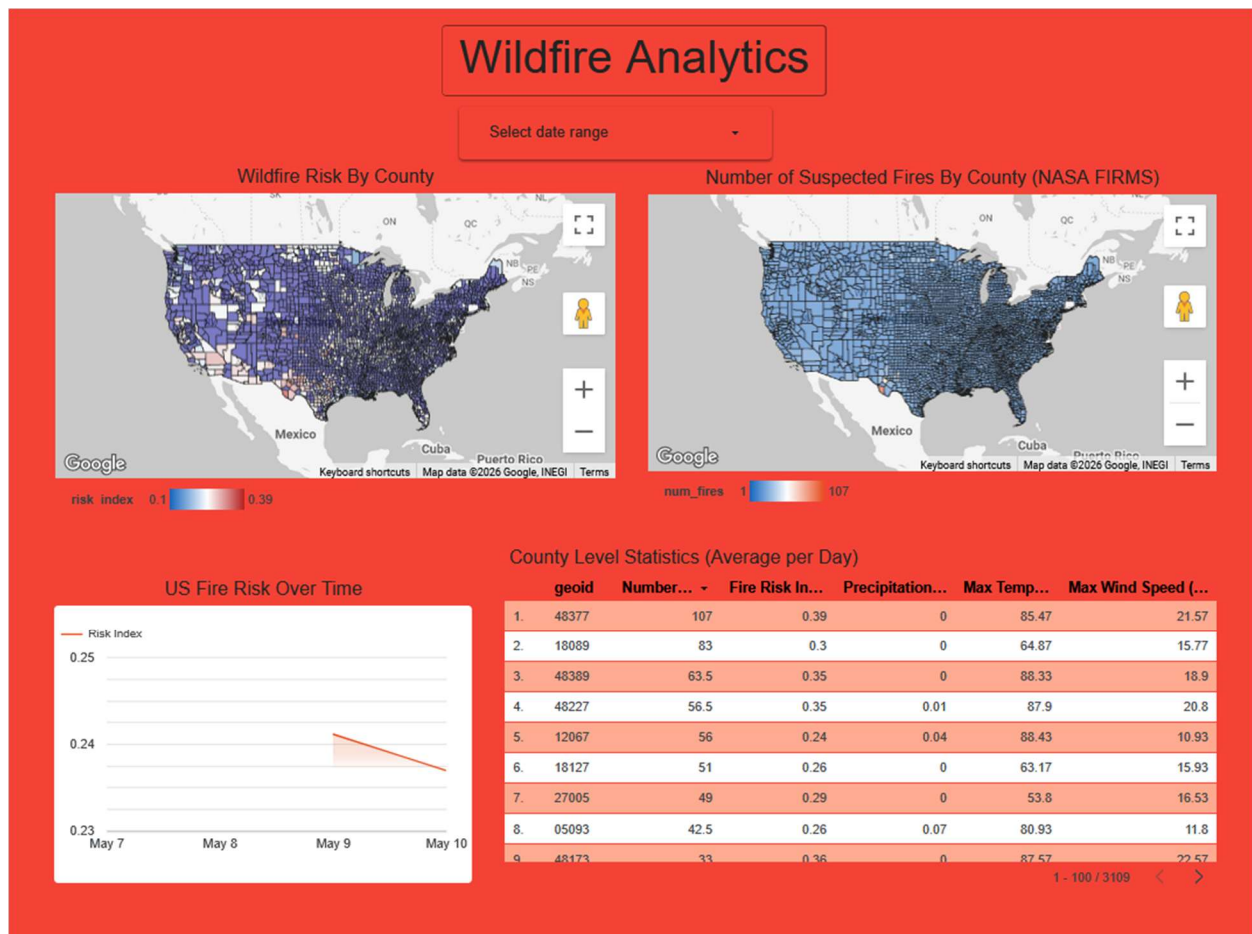
CASE
  WHEN num_fires IS NULL
    OR avg_bright IS NULL
    OR avg_frp IS NULL
  THEN NULL
  ELSE ROUND(
    0.20 * n_temp_max
    + 0.20 * n_wind_gust
    + 0.10 * n_sunshine
    + 0.10 * n_fires
    + 0.05 * n_frp
    + 0.05 * n_bright
    - 0.20 * n_rain
    - 0.10 * n_temp_min
    , 4)
END AS risk_index
FROM normalized

```

Results Obtained

As described above the table produced after all of the data processing contained cleaned data that combined weather and wildfire data at the county level. This allows for data visualization that can combine the two for a more nuanced user experience. I created a fully automated data pipeline that goes from daily API calls to an updated analytics dashboard, which is the most realistic real world build out for a project of this subject.

Interactive Dashboard



This Looker Studio dashboard was created to give users a useful tool to understand wildfire trends in the US. It provides two main maps, one showing a wildfire risk index for each US county and another showing average detected wildfires in each county. A date control is included to allow users to select over a date range. Also included for the users benefit is a overall US fire risk time series graph which shows seasonal differences in fire risk. A table is also provided to give a breakdown of statistics for each specific county, including daily weather statistics.

A link to the dashboard is provided here:

<https://datastudio.google.com/reporting/206c1783-3d28-41ad-b1f0-ecb42fdb26f4>

Challenges Encountered

One of the biggest challenges I encountered in this project was that Looker Studio did not support my original ideas for dividing up the geographical data. I originally wanted to divide up

the US map into hexagons, which is convenient because the size of the hexagons can be controlled. This proved to be too complex for implementation in Dataproc, so I decided to utilize a different approach. I created a simple set of geographical boxes to divide up the map. However, after I had gone through the data ingestion and successfully created a Dataproc script for this example, I realized that Looker Studio does not support customer geographical boundaries like the ones I had created. So, my final decision was to redo the Cloud Run service for the Open-Meteo data to pull one daily weather point for each county, and then I created a Dataproc job to calculate which county each suspected wildfire was in. the Dataproc job proved to be especially challenging, but I eventually got it working.

The biggest challenge I had overall was getting the Cloud Run, Cloud Scheduler, and Dataproc Workflow jobs to run smoothly, because setting them up properly so they did not break was very challenging and doing testing was very tedious. I eventually resolved this through many hours of work making small adjustments to source code and job settings, but it was incredibly frustrating.

Lessons Learned

Initially, I was very ambitious in my goals for this project, which included pulling historic data to perform Machine Learning and provide a historic understanding of normal weather patterns, but I quickly realized that this was far too ambitious for the time frame I had. This was, in majority do to the complexity of implementing dynamic data pipelines where data updates are being made. While creating a pipeline to handle dynamic dataflows was very challenging, it helped me understand why Google's Cloud system is setup to be modular, which helps make complex data pipelines understandable and more testable. I also developed a deep understanding of Cloud Run and Cloud Scheduler since these two services were used many times in this project.

Outside of my personal development in topics from this course, I also found that planning a project out in depth before actually getting hands on with code and deployment is very important. If I had looked into what geographical shapes were supported by Looker Studio prior to attempting to implement the hexagons and squares, I would have saved an immense amount of time.

Next Steps If More Time Were Available

- Ingest historic wildfire and weather data for both machine learning development and historical understanding of what is “normal” for geographical locations.
- Upgrade Open-Meteo API subscriptions from free subscription to get more API calls for both hourly updates and finer grain weather data.
- Find wildfire API that provides actual wildfire maps with fire geographical boundaries.
- I also would want to build more trigger-based Cloud Run services, since they seem to be more robust than simply scheduling services after each other that are dependent on another service.
- Create more of a streaming data pipeline which is more realistic than a batch data pipeline in modern systems.